

VERO STATUS 4.0.0 - COMPLETE SOURCE FOR REBUILD

Jira ASPF-1578 · Quality AI Automation · a document only: nothing in it runs

What this is: the full source (24 text files) of "Vero Status", a small program that shows in the browser whether Vero is available and which model it uses. Python 3.8+ standard library only. The guide PDF is sent separately.

INSTRUCTIONS FOR THE AI ASSISTANT (Vero CLI or other) - recreate the project folder EXACTLY:

- 1. Create the folder vero-status in the location the user gives (default C:\Tools\).
- 2. Every file starts with a row "=== FILE: <path> (<n> lines) ===" and ends with "=== END FILE: <path> ===".
- 3. Rows between them encode the file, one source line per numbered row:
 - " 12 | content" -> line 12. Every `␣` (open box) in the content is ONE SPACE. It marks leading spaces, runs of 2+ spaces and spaces at a row edge; single spaces between words are printed normally. The `␣` character never occurs in the real files.
 - " 12 |" -> line 12 is an empty line.
 - " + more" -> continuation of the previous line: append "more" directly (no space, no newline). Ignore page headers/footers ("Vero Status ... source for rebuild", "page N"). Line numbers must run 1..n without gaps; n is in the FILE row. Every file ends with a newline.
- 4. Write files as UTF-8 without BOM, LF line endings. Do not reformat, translate, summarise or "fix" anything.
- 5. Create verify_rebuild.py from the section VERIFY SCRIPT at the end (same row rules), OUTSIDE the project folder, and run: `py verify_rebuild.py C:\Tools\vero-status`
It checks every file's SHA-256 and sets CRLF line endings where Windows needs them (.cmd, START_HERE.txt). Recreate any file reported MISSING or DIFFERENT, then run it again until it prints "OK: all files match".
- 6. Run the tests inside the folder: `py -m unittest discover -s tests -t .` Expected: OK (30 tests).
- 7. Tell the user it is ready: double-click "Vero Status.pyw" to open the dashboard (see START_HERE.txt).
Do not start the program or call the real Vero CLI unless the user asks.

FILE LIST (path · lines · line endings · SHA-256 of the content with LF endings)

.gitattributes	7	LF	efa38e63b1747daed572feeff39747d40fd4cfa3a4568b11815cb0bf24332d96
.gitignore	8	LF	b592d17a523d7d8d8bd950419afdeb5f502d4aedd8b65a2887bd71f0c514aa33
CHANGELOG.md	14	LF	ffa4ff4eef221d1e49d7726d5310e8c1e87ec56bdf0e5d6ece6694b11ef5a5dd
README.md	44	LF	d8b413eae23a6ecad126fcb99da6af6dda6912e1f37cd0983fe75b850b683970
START_HERE.txt	20	CRLF	e3e61140954e96168ca25e366ccb9384c45ef2a66fc04baa2f845eaf2da4ab25
VERO_CONTEXT.md	80	LF	057690356ec1dc3d26ce9fe9384bf82fcbb79c5b721158e9a1e1a452addfa50
Vero Status.pyw	10	LF	d225f8171b6196c264964f66cba8c2992695a171db6c2f047f9c694fb8358197
pyproject.toml	22	LF	0490be6ed05d10f4fe955a49fe965dd14698e32c6292a00707776eac1b54f9fd4
scripts/build_source_pdf.py	224	LF	1150c226ffe2449688aedc50c56a1404a28958e62668dced9418ea684cb58d91
scripts/fake_vero.py	56	LF	4908eb33e4a64621f955e551ef0efce2e644ca3a6ba51e9c202b80cf01d35ae8
scripts/vero	2	LF	45890958a3fcde4e02f1d4c41412869e6f71a9efe2d2d237cd54cf8d84e001fa
scripts/vero.cmd	2	CRLF	15e9a293d205d911b2e2746af1c4361906c0b4138dc111be5acc2a653a6cd0ca
tests/__init__.py	1	LF	8d8ab1348161355b2c6856609575f5becac99c397071c7dcdad95d00597f39fb
tests/helpers.py	22	LF	dfd00de3c741b7782d8ce5eeaad15ac3534a69b8a7b5dfcbe3fb9d26056ac971
tests/test_monitor_server.py	207	LF	4d0c221bbf6b72efae3d736d309368109978227ef2d448295efa034c6f553f75
tests/test_settings.py	45	LF	43a3cbf79366e7f16dce3847e58d5b74df54570b570c6e3408c95961554c1303
tests/test_vero.py	98	LF	00f1697582183a59e1b0c852dd3f37bbd6fa8d4a0df42ae1294d3b2cde09a44e
vero_status/__init__.py	4	LF	8770ca38a34434db168778fe3e48726bf0c041c78e757c1b2df1f37fdf7e1e1a
vero_status/app.py	84	LF	e1149390fa4df886d89d37f0fd6f4be93ca73f383e47b8e6b8196711764641b0
vero_status/monitor.py	198	LF	2b353f1c385be006eee7e7114b3daac103140fc647f4f04377bf86626db28472
vero_status/server.py	104	LF	4fe22178766d0357e85bb164bbcfbac66555c8914a9b8b68b026e2365c3cc851
vero_status/settings.py	62	LF	61d2441d9c9da8c04e624cbc815a299333724c87786eb6f8b3b6fe646f19beb4
vero_status/vero.py	200	LF	31fa059c4bf44a2dadca0ab171b4384bbe4d98ec1a0e6428a13bac5d18d17025
vero_status/web/index.html	250	LF	570767ef4b1a040b8ea654d2427b4035ee878cb4b3b32c0fc04afba2e28e2561

=== FILE: .gitattributes (7 lines) ===

```
1 | * text=auto
2 | *.py text eol=lf
3 | *.pyw text eol=lf
4 | *.md text
5 | *.cmd text eol=crlf
6 | START_HERE.txt text eol=crlf
7 | scripts/vero text eol=lf
=== END FILE: .gitattributes ===
```

=== FILE: .gitignore (8 lines) ===

```
1 | __pycache__/  
2 | *.py[cod]  
3 | data/  
4 | dist/  
5 | .venv/  
6 | .mypy_cache/  
7 | .ruff_cache/  
8 | *.egg-info/
```

=== END FILE: .gitignore ===

```
=== FILE: CHANGELOG.md (14 lines) ===
```

```
1 | # Changelog
2 |
3 | ## 4.0.0
4 | Rewritten as a simple program with a dashboard (feedback on 3.0: too many command windows and steps).
5 | - Double-click `Vero Status.pyw`: no console, no setup, no Task Scheduler. A second double-click just opens the dashboard.
6 | - The dashboard shows whether Vero is available and **which model Vero uses**: the configured model (`vero config`)
7 |   and the model asked in the last check (`modelInfo`). Also the Vero version, response time, 24 h availability and the last checks.
8 | - Check now, Settings (interval: off / 5 / 15 / 30 / 60 min, check model, timeout, Vero location) and Quit on the page.
9 | - Keeps the safe parts of 3.0: `--json` stream with `completion_result` only, no `--yolo`, empty working folder,
10 |  the whole process tree killed on timeout, Host and Origin checks, nothing secret stored. Errors at start show a Windows message box.
11 | - 30 tests (fake Vero), ruff, mypy strict for Linux and Windows, checked in a real browser.
12 |
13 | ## 3.0.0 and older
14 | Vero Availability Monitor (3.0) and Vero Latency Monitor (0.x): command-line tools with installer scripts. Superseded.
=== END FILE: CHANGELOG.md ===
```

```
=== FILE: README.md (44 lines) ===
```

```

1 | # Vero Status
2 |
3 | Double-click Vero Status and a dashboard opens in your browser showing:
4 |
5 | - Is Vero available? Big green or red status, since when, and the reason if not.
6 | - Which model does Vero use? The model Vero is configured with ('vero config'), and the model that answered the last check.
7 | - The Vero CLI version, how long the last answer took, availability over the last 24 h, and the last checks.
8 |
9 | Buttons: Check now, Settings (how often to check, which model to use for the check, Vero location), Quit.
10 |
11 | Jira ASPF-1578 · Version 4.0.0 · Python 3.8+ standard library only, no install, no command window.
12 |
13 | ## Use it
14 |
15 | 1. Put the folder somewhere fixed, e.g. 'C:\Tools\vero-status'.
16 | 2. Double-click Vero Status.pyw. The dashboard opens at http://127.0.0.1:8767.
17 | 3. Optional: right-click 'Vero Status.pyw' and choose Send to > Desktop (create shortcut).
18 |    To start it with Windows, put that shortcut in the folder that opens with 'Win+R' -> 'shell:startup'.
19 |
20 | Double-clicking again while it runs just opens the dashboard. Quit stops it.
21 |
22 | ## How a check works
23 |
24 | 1. 'vero version': is Vero installed?
25 | 2. 'vero config': which model and provider Vero is set to (free).
26 | 3. 'vero task --json -t 120 -c <empty folder> -m <check model> "Reply with exactly: OK"'.
27 |    Vero is available when the 'completion_result' answer arrives. This costs 2 small model calls and takes about 1 minute.
28 |    The check model defaults to the cheap Haiku 4.5; leave it empty in Settings to test Vero's own model.
29 |
30 | The checks never use '--yolo', run in an empty folder, and never store answers or secrets.
31 | The data lives in the program's 'data/' folder ('settings.json', 'history.json', 'vero-status.log').
32 |
33 | ## Try it without Vero
34 |
35 | In Settings, set Vero CLI location to 'scripts\vero.cmd' (Windows) or 'scripts/vero' (Linux/macOS; run 'chmod +x scripts/vero' once): a fake Vero that answers like the real one.
36 |
37 | ## Develop
38 |
39 | ```
40 | python -m unittest discover -s tests -t .# 30 tests, fake Vero only
41 | uvx ruff check . && uvx ruff format --check .# lint + format
42 | uvx mypy && uvx mypy --platform win32# strict typing
43 | uv run --no-project --with reportlab python scripts/build_source_pdf.py# source as PDF for rebuild
44 | ```

```

```
=== END FILE: README.md ===
```

```
=== FILE: START_HERE.txt (20 lines) ===
```

```

1 | VERO STATUS 4 (ASPF-1578)
2 | =====
3 |
4 | Shows in your browser whether Vero is available and which model it uses.
5 | Needs Python 3.8+ only. Nothing to install. No command window.
6 |
7 | 1. Put this folder somewhere fixed, e.g. C:\Tools\vero-status
8 |
9 | 2. Double-click "Vero Status.pyw"
10 | Your browser opens the dashboard (http://127.0.0.1:8767). The first check takes about a minute.
11 |
12 | 3. Optional: right-click "Vero Status.pyw" -> Send to -> Desktop (create shortcut).
13 | To start it with Windows: Win+R -> shell:startup -> put that shortcut there.
14 |
15 | In the dashboard: Check now | Settings (how often, which model, Vero location) | Quit
16 |
17 | If an older version is installed (Vero Latency Monitor or Vero Availability Monitor), run
18 | uninstall.bat in its folder so it stops calling Vero.
19 |
20 | Problems: open data\vero-status.log, or read Vero_Status_Guide.pdf.
=== END FILE: START_HERE.txt ===
```

```
=== FILE: VERO_CONTEXT.md (80 lines) ===
```

```

1 | # VERO_CONTEXT: Vero Status 4.0.0
2 |
3 | > **For the AI reading this:** this file describes the whole program. Read a file before changing it, and keep the rules at the end.
4 |
5 | ## What it is
6 |
7 | A small program for one laptop. When you double-click `Vero Status.pyw`, it opens a dashboard in the browser at `http://127.0.0.1:8767`. The dashboard shows:
8 |
9 | - whether **Vero is available** (Vero CLI 2.3.x);
10 | - **which model Vero uses**: the configured model from `vero config`, and the model the last check used (`modelInfo` in the `--json` stream);
11 | - the Vero version, the response time, availability over 24 h, and the last 48 checks;
12 | - buttons for Check now, Settings and Quit.
13 |
14 | It uses only the Python 3.8+ standard library. It opens no console window (`pythonw`), and every child process starts with `CREATE_NO_WINDOW`.
15 |
16 | The Jira story is ASPF-1578, for the Quality AI Automation team at NXP. Version 4.0.0 replaces the 3.0 "Vero Availability Monitor", which was command-line based.
17 |
18 | ## Files
19 |
20 | ```
21 | Vero Status.pyw#####double-click entry: adds the folder to sys.path, calls vero_status.app.run()
22 | vero_status/
23 | __init__.py#####__version__ = "4.0.0", APP_ID = "vero-status"
24 | __app.py#####single instance (GET /api/health), logging to data/vero-status.log, opens the browser,
25 | #####shows a Windows message box on errors (no console)
26 | __vero.py#####find_vero, run (no window, kills the process tree on timeout), and pure parsers:
27 | #####parse_version, parse_config, parse_task_stream, task_args, clean (masks credentials)
28 | __monitor.py#####perform_check(settings, workdir, run) -> record; Monitor: timer thread, check_now,
29 | #####history (data/history.json, last 500), update_settings, state() for the page
30 | __settings.py#####Settings (frozen dataclass), validation, load/save of data/settings.json
31 | __server.py#####127.0.0.1 only. GET / , /api/state, /api/health; POST /api/check, /api/settings, /api/quit.
32 | #####Host and Origin are checked.
33 | __web/index.html#####the dashboard (one file, vanilla JS, light and dark, polls /api/state)
34 | scripts/fake_vero.py#####stand-in Vero: version, config, task --json (FAKE_VERO_MODE ok|down|auth|hang|nocompletion|refuse)
35 | scripts/vero, vero.cmd#####launch the fake Vero (point "Vero CLI location" at one of these for a demo)
36 | scripts/build_source_pdf.py#####writes the source as a PDF for rebuilding (developer tool, needs reportlab)
37 | tests/#####unittest; never calls the real Vero
38 | ```
39 |
40 | ## One check (`monitor.perform_check`)
41 |
42 | 1. `find_vero(settings.vero_path)` looks for the Vero CLI in this order: the configured path, `vero` on the PATH (`vero.cmd` on Windows), then `%APPDATA%\npm\vero.cmd`. If nothing is found, the r
+ result is "not available" with the reason "not found".
43 | 2. `vero version`: if this fails, the result is "not available".
44 | 3. `vero config` reads the settings `actModeApiModelId`, `actModeApiProvider` and `awsRegion`, given either as `key : value` lines or as JSON. If this step fails, the configured model is simply u
+ nknown; it does not affect the status.
45 | 4. `vero task --json -t <timeout> -c data/sandbox [-m <check_model>]` "Reply with exactly: OK", killed at timeout + 30 s.
46 | --- **Available** if and only if a `completion_result` event arrives. A different answer still counts as available, with a note.
47 | --- `partial` events are ignored. `api_req_started`, which holds the full prompt, is never kept.
48 | --- The record holds `time`, `available`, `reason`, `seconds` (only when there is an answer), `asked_model`, `provider`, `vero_version`, `configured` `{model, provider, region}` and `trigger
+ ` (start, auto or manual).
49 |
50 | ## Settings
51 |
52 | Stored in `data/settings.json` and editable on the page:
53 |
54 | | Setting | Default | Allowed values |
55 | |---|---|---|
56 | | `vero_path` | `` (find it automatically) | any path |
57 | | `check_model` | Haiku 4.5 | a model id, or `` to use Vero's own model |
58 | | `interval_minutes` | 15 | 0 (off), 5, 15, 30, 60 |
59 | | `timeout_seconds` | 120 | 30-600 |
60 | | `port` | 8767 | 1024-65535; change it in the file only |
61 |
62 | A broken settings file falls back to the defaults.
63 |

```

```

64 | ## Quality gates
65 |
66 | ```
67 | python -m unittest discover -s tests -t .(30 tests, Python 3.8–3.13)
68 | uvx ruff check . && uvx ruff format --check .
69 | uvx mypy && uvx mypy --platform win32(strict)
70 | ```
71 |
72 | ## Rules
73 |
74 | 1. Keep it simple for the user: double-click, a dashboard, no command windows, no installer.
75 | 2. Standard library only. No CDN and no external scripts in the page.
76 | 3. Never pass `--yolo`. Run the task in the empty `data/sandbox` folder. On timeout, kill the whole process tree.
77 | 4. Store no answers, no secrets and no `api_req_started` content. Mask credentials in error text.
78 | 5. Bind to 127.0.0.1 only, and keep the Host and Origin checks on every request.
79 | 6. Tests use the fake Vero only.
80 | 7. Commit messages and PR titles have the form `ASPF-1578: <summary>`. Update the CHANGELOG and this file when behaviour changes.
=== END FILE: VERO_CONTEXT.md ===

```

```
=== FILE: Vero Status.pyw (10 lines) ===
1 | """Double-click to open the Vero Status dashboard (no console window)."""
2 |
3 | import sys
4 | from pathlib import Path
5 |
6 | sys.path.insert(0, str(Path(__file__).resolve().parent))
7 |
8 | from vero_status.app import run_# noqa: E402
9 |
10 | run()
=== END FILE: Vero Status.pyw ===
```

```
=== FILE: pyproject.toml (22 lines) ===
```

```
1 | [project]
2 | name = "vero-status"
3 | version = "4.0.0"
4 | description = "Dashboard: is Vero available, and which model does it use? (ASPF-1578)"
5 | requires-python = ">=3.8"
6 | dependencies = []
7 |
8 | [tool.ruff]
9 | line-length = 130
10 | target-version = "py38"
11 |
12 | [tool.ruff.lint]
13 | select = ["E", "F", "W", "I", "B", "UP", "S", "SIM", "RUF"]
14 | ignore = ["S603", "S607", "S310", "UP006", "UP007", "UP035", "UP045", "RUF001", "RUF003"]
15 |
16 | [tool.ruff.lint.per-file-ignores]
17 | "tests/**" = ["S101", "S108"]
18 |
19 | [tool.mypy]
20 | python_version = "3.9"
21 | files = ["vero_status"]
22 | strict = true
```

```
=== END FILE: pyproject.toml ===
```

```
=== FILE: scripts/build_source_pdf.py (224 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Build dist/Vero_Status_<version>_Source.pdf: the complete source as a PDF document.
3 |
4 | For channels that accept only documents. Developer tool (needs reportlab):
5 | uv run --no-project --with reportlab python scripts/build_source_pdf.py
6 |
7 | Listing format (survives PDF text extraction, which drops blank lines and collapses spaces):
8 | """
9 | line 12 | content"
10 | line 12 of the file; every MARK (U+2423, open box) in content is one space
11 | (used for leading spaces, runs of 2+ spaces and spaces at a row edge)
12 | line 12 | "
13 | line 12 is empty
14 | "
15 | + more"
16 | continuation of the previous line (joined with nothing in between)
17 | The MARK character never appears in the sources (checked at build time).
18 | """
19 |
20 | import hashlib
21 | import re
22 | import sys
23 | from pathlib import Path
24 |
25 | from reportlab.lib.pagesizes import A4, landscape
26 | from reportlab.pdfbase import pdfmetrics
27 | from reportlab.pdfbase.ttfonts import TTFont
28 | from reportlab.pdfgen import canvas
29 |
30 | ROOT = Path(__file__).resolve().parents[1]
31 | sys.path.insert(0, str(ROOT))
32 | from vero_status import __version__ # noqa: E402
33 |
34 | SKIP_DIRS = {
35 |     "data",
36 |     "data-demo",
37 |     "reports",
38 |     "dist",
39 |     "__pycache__",
40 |     ".pytest_cache",
41 |     ".mypy_cache",
42 |     ".ruff_cache",
43 |     ".venv",
44 |     "venv",
45 | }
46 | SKIP_SUFFIXES = (".pyc", ".log", ".egg-info")
47 | SKIP_FILES = {"docs/Vero_Status_Guide.html"}
48 | CRLF = (".bat", ".cmd")
49 | CRLF_FILES = {"START_HERE.txt"}
50 | FONT_PATHS = ["/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", "C:/Windows/Fonts/consola.ttf"]
51 | SIZE, LEADING, MARGIN = 6.4, 7.7, 28
52 | PAGE_W, PAGE_H = landscape(A4)
53 | MARK = "\u2423" # open box, shown for a space that extraction could lose
54 |
55 | def files():
56 |     for f in sorted(ROOT.rglob("*")):
57 |         rel = f.relative_to(ROOT)
58 |         if (
59 |             f.is_dir()
60 |             or SKIP_DIRS & set(rel.parts)
61 |             or any(part.endswith(SKIP_SUFFIXES) for part in rel.parts)
62 |             or rel.as_posix() in SKIP_FILES
63 |         ):
64 |             continue
65 |         yield rel.as_posix(), f.read_bytes().decode("utf-8").replace("\r\n", "\n")
66 |
67 | def lf_sha(text: str) -> str:
68 |     return hashlib.sha256(text.encode("utf-8")).hexdigest()

```

```

67 |
68 | def protect(chunk: str) -> str:
69 |     """Mark every space an extractor could drop: leading, trailing, and runs of 2+."""
70 |     chunk = re.sub(r" {2,}", lambda m: MARK * len(m.group()), chunk)
71 |     if chunk.startswith(" "):
72 |         chunk = MARK + chunk[1:]
73 |     if chunk.endswith(" "):
74 |         chunk = chunk[:-1] + MARK
75 |     return chunk
76 |
77 |
78 | def encode(lines, width):
79 |     """Source lines -> listing rows (see module docstring)."""
80 |     rows = []
81 |     for n, line in enumerate(lines, 1):
82 |         head = f"{n:>4} |"
83 |         if not line:
84 |             rows.append(head)
85 |             continue
86 |         chunks = [line[i : i + width] for i in range(0, len(line), width)]
87 |         rows.append(f"{head} {protect(chunks[0])}")
88 |         rows.extend(f"      {protect(c)}" for c in chunks[1:])
89 |     return rows
90 |
91 |
92 | VERIFY_TEMPLATE = """import hashlib, pathlib, sys
93 | # usage: py verify_rebuild.py C:\\Tools\\vero-status
94 | root = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "vero-status")
95 | CRLF = {crlf!r}
96 | FILES = {{
97 |     {entries}}}
98 | bad = 0
99 | for path, sha in FILES.items():
100 |     p = root / path
101 |     if not p.exists():
102 |         print("MISSING", path); bad += 1; continue
103 |     text = p.read_bytes().decode("utf-8").replace("\\r\\n", "\\n")
104 |     if hashlib.sha256(text.encode("utf-8")).hexdigest() != sha:
105 |         print("DIFFERENT", path); bad += 1; continue
106 |     want = text.replace("\\n", "\\r\\n") if path in CRLF else text
107 |     if p.read_bytes() != want.encode("utf-8"):
108 |         p.write_bytes(want.encode("utf-8")); print("fixed line endings", path)
109 |     print("OK: all files match" if not bad else str(bad) + " file(s) wrong: recreate them from the document")
110 |     sys.exit(1 if bad else 0)
111 | """
112 |
113 |
114 | class Doc:
115 |     def __init__(self, path: Path, font: str):
116 |         self.c = canvas.Canvas(str(path), pagesize=(PAGE_W, PAGE_H))
117 |         self.c.setTitle(f"Vero Status {__version__} - complete source for rebuild")
118 |         self.c.setAuthor("Quality AI Automation (ASPF-1578)")
119 |         self.c.setSubject("Source listing. A document only: nothing in it runs.")
120 |         self.font = font
121 |         self.page = 0
122 |         self.y = 0.0
123 |         self.new_page()
124 |
125 |     def new_page(self):
126 |         if self.page:
127 |             self.c.showPage()
128 |             self.page += 1
129 |             self.c.setFont(self.font, 6)
130 |             self.c.setFillGray(0.45)
131 |             self.c.drawString(MARGIN, PAGE_H - 16, f"Vero Status {__version__} · source for rebuild · ASPF-1578")
132 |             self.c.drawRightString(PAGE_W - MARGIN, 12, f"page {self.page}")
133 |             self.c.setFillGray(0)

```

```

134 |         self.y = PAGE_H - MARGIN - 8
135 |
136 |     def row(self, text: str, size: float = SIZE, gray: float = 0):
137 |         if self.y < MARGIN:
138 |             self.new_page()
139 |             self.c.setFont(self.font, size)
140 |             self.c.setFillGray(gray)
141 |             self.c.drawString(MARGIN, self.y, text)
142 |             self.y -= max(LEADING, size * 1.25)
143 |
144 |     def gap(self, n: float = 1):
145 |         self.y -= LEADING * n
146 |
147 |     def save(self):
148 |         self.c.save()
149 |
150 |
151 | def main() -> None:
152 |     font_path = next((p for p in FONT_PATHS if Path(p).exists()), None)
153 |     if not font_path:
154 |         sys.exit("no monospace TTF font found (DejaVu Sans Mono or Consolas)")
155 |     pdfmetrics.registerFont(TTFont("Mono", font_path))
156 |     char_w = pdfmetrics.stringWidth("M", "Mono", SIZE)
157 |     width = int((PAGE_W - 2 * MARGIN) / char_w) - 8 # minus the "NNNN | " gutter
158 |     items = list(files())
159 |     bad = [p for p, t in items if MARK in t]
160 |     if bad:
161 |         sys.exit(f"MARK character found in {bad}: the listing would be ambiguous")
162 |     crlf = sorted(p for p, _ in items if p.endswith(CRLF) or p in CRLF_FILES)
163 |     out = ROOT / "dist" / f"Vero_Status_{__version__}_Source.pdf"
164 |     out.parent.mkdir(exist_ok=True)
165 |     d = Doc(out, "Mono")
166 |
167 |     d.row(f"VERO STATUS {__version__} - COMPLETE SOURCE FOR REBUILD", 13)
168 |     d.row("Jira ASPF-1578 · Quality AI Automation · a document only: nothing in it runs", 8, 0.35)
169 |     d.gap()
170 |     intro = f"""\
171 | What this is: the full source ({len(items)} text files) of "Vero Status", a small program that shows in the browser whether
172 | Vero is available and which model it uses. Python 3.8+ standard library only. The guide PDF is sent separately.
173 |
174 | INSTRUCTIONS FOR THE AI ASSISTANT (Vero CLI or other) - recreate the project folder EXACTLY:
175 | 1. Create the folder vero-status in the location the user gives (default C:\\Tools\\).
176 | 2. Every file starts with a row{u}""== FILE: <path> (<n> lines) ""== and ends with{u}""== END FILE: <path> ""==.
177 | 3. Rows between them encode the file, one source line per numbered row:
178 | {u}12 | content{u}-> line 12. Every {MARK} (open box) in the content is ONE SPACE. It marks leading
179 | spaces, runs of 2+ spaces and spaces at a row edge; single spaces between words
180 | are printed normally. The {MARK} character never occurs in the real files.
181 | {u}12 | ""{u}-> line 12 is an empty line.
182 | {u}+ more{u}-> continuation of the previous line: append "more" directly (no space, no newline).
183 | Ignore page headers/footers ("Vero Status ... source for rebuild", "page N").
184 | Line numbers must run 1..n without gaps; n is in the FILE row. Every file ends with a newline.
185 | 4. Write files as UTF-8 without BOM, LF line endings. Do not reformat, translate, summarise or "fix" anything.
186 | 5. Create verify_rebuild.py from the section VERIFY SCRIPT at the end (same row rules), OUTSIDE the project
187 | folder, and run: {u}py verify_rebuild.py C:\\Tools\\vero-status
188 | It checks every file's SHA-256 and sets CRLF line endings where Windows needs them (.cmd, START_HERE.txt).
189 | Recreate any file reported MISSING or DIFFERENT, then run it again until it prints "OK: all files match".
190 | 6. Run the tests inside the folder: {u}py -m unittest discover -s tests -t {u}Expected: OK (30 tests).
191 | 7. Tell the user it is ready: double-click "Vero Status.pyw" to open the dashboard (see START_HERE.txt).
192 | Do not start the program or call the real Vero CLI unless the user asks.
193 | """
194 |     for line in intro.splitlines():
195 |         d.row(line, 7.2)
196 |         d.gap()
197 |     d.row("FILE LIST (path · lines · line endings · SHA-256 of the content with LF endings)", 7.2)
198 |     for p, t in items:
199 |         lines = t.split("\n")[:-1] if t.endswith("\n") else t.split("\n")
200 |         d.row(f'{u}{p[:52]} {len(lines):>5}{u}' 'CRLF' if p in crlf else 'LF{u}'{lf_sha(t)}', 6.2)

```

```

201 |
202 |     for p, t in items:
203 |         lines = t.split("\n")[:-1] if t.endswith("\n") else t.split("\n")
204 |         d.new_page()
205 |         d.row(f"=== FILE: {p} ({len(lines)} lines) ===", 7.5)
206 |         for r in encode(lines, width):
207 |             d.row(r)
208 |         d.row(f"=== END FILE: {p} ===", 7.5)
209 |
210 |     entries = "".join(f"{p!r}: {lf_sha(t)!r},\n" for p, t in items)
211 |     verify = VERIFY_TEMPLATE.format(crlf=crlf, entries=entries)
212 |     d.new_page()
213 |     vlines = verify.split("\n")[:-1]
214 |     d.row(f"=== VERIFY SCRIPT: verify_rebuild.py ({len(vlines)} lines) ===", 7.5)
215 |     for r in encode(vlines, width):
216 |         d.row(r)
217 |     d.row("=== END VERIFY SCRIPT ===", 7.5)
218 |     d.save()
219 |     (ROOT / "dist" / "verify_rebuild.py").write_text(verify, encoding="utf-8")
220 |     print(f"wrote {out} ({len(items)} files, {d.page} pages, {out.stat().st_size // 1024} KB, {width} chars per row)")
221 |
222 |
223 | if __name__ == "__main__":
224 |     main()
=== END FILE: scripts/build_source_pdf.py ===

```

```
=== FILE: scripts/fake_vero.py (56 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Stand-in for the Vero CLI (demo and tests): `version`, `config`, `task --json ...` like Vero CLI 2.3.x.
3 |
4 | FAKE_VERO_MODE: ok (default) | down | auth | hang | nocompletion | refuse
5 | FAKE_VERO_DELAY: seconds before answering (default 2)
6 | """
7 |
8 | import json
9 | import os
10 | import sys
11 | import time
12 |
13 |
14 | def emit(obj):
15 |     print(json.dumps(obj), flush=True)
16 |
17 |
18 | def main(argv):
19 |     cmd = argv[0] if argv else "version"
20 |     if cmd in ("version", "--version"):
21 |         print("vero 2.3.3 (fake0000)")
22 |         return 0
23 |     if cmd == "config":
24 |         print("actModeApiProvider : bedrock")
25 |         print("actModeApiModelId: us.anthropic.claude-opus-5")
26 |         print("awsRegion: us-west-2")
27 |         return 0
28 |     if cmd not in ("task", "t"):
29 |         print(f"unknown command {cmd}", file=sys.stderr)
30 |         return 2
31 |     args = argv[1:]
32 |     model = args[args.index("-m") + 1] if "-m" in args else "us.anthropic.claude-opus-5"
33 |     info = {"providerId": "bedrock", "modelId": model, "mode": "act"}
34 |     mode = os.environ.get("FAKE_VERO_MODE", "ok")
35 |     delay = float(os.environ.get("FAKE_VERO_DELAY", "2"))
36 |     emit({"type": "task_started", "taskId": str(int(time.time() * 1000))})
37 |     emit({"type": "say", "say": "task", "text": args[-1], "modelInfo": info})
38 |     emit({"type": "say", "say": "api_req_started", "text": "SYSTEM PROMPT FAKE-SECRET-PROMPT", "modelInfo": info})
39 |     if mode == "hang":
40 |         time.sleep(3600)
41 |     if mode == "auth":
42 |         print("error: Unauthorized token=abc123secret", file=sys.stderr)
43 |         return 1
44 |     if mode == "down":
45 |         print("error: provider unavailable (connect ETIMEDOUT)", file=sys.stderr)
46 |         return 1
47 |     time.sleep(delay)
48 |     answer = "I cannot do that." if mode == "refuse" else "OK"
49 |     emit({"type": "say", "say": "text", "text": answer[1:], "partial": True, "modelInfo": info})
50 |     if mode != "nocompletion":
51 |         emit({"type": "say", "say": "completion_result", "text": answer, "modelInfo": info})
52 |     return 0
53 |
54 |
55 | if __name__ == "__main__":
56 |     sys.exit(main(sys.argv[1:]))
=== END FILE: scripts/fake_vero.py ===
```

```
=== FILE: scripts/vero (2 lines) ===
  1 | #!/bin/sh
  2 | exec python3 "${dirname "$0"}/fake_vero.py" "$@"
=== END FILE: scripts/vero ===
```

```
=== FILE: scripts/vero.cmd (2 lines) ===
  1 | @echo off
  2 | py -3 "%~dp0fake_vero.py" %*
=== END FILE: scripts/vero.cmd ===
```

```
=== FILE: tests/__init__.py (1 lines) ===
  1 | """Tests (never call the real Vero)."""
=== END FILE: tests/__init__.py ===
```

```
=== FILE: tests/helpers.py (22 lines) ===
1 | """Shared helpers: run the bundled fake Vero CLI through the real process code."""
2 |
3 | import sys
4 | from pathlib import Path
5 |
6 | ROOT = Path(__file__).resolve().parents[1]
7 | sys.path.insert(0, str(ROOT))
8 |
9 | from vero_status import vero_# noqa: E402
10 |
11 | FAKE = ROOT / "scripts" / "fake_vero.py"
12 |
13 |
14 | def fake_run(args, timeout_s, cwd=None):
15 |     """Like vero.run, but 'vero' is the fake script started with this Python."""
16 |     return vero.run([sys.executable, str(FAKE), *args[1:]], timeout_s, cwd)
17 |
18 |
19 | def fake_settings(**over):
20 |     from vero_status.settings import Settings
21 |
22 |     return Settings(vero_path=sys.executable, **over)
=== END FILE: tests/helpers.py ===
```

```
=== FILE: tests/test_monitor_server.py (207 lines) ===
```

```

1 | import json
2 | import os
3 | import socket
4 | import tempfile
5 | import threading
6 | import time
7 | import unittest
8 | import urllib.error
9 | import urllib.request
10 | from pathlib import Path
11 | from unittest import mock
12 |
13 | from tests.helpers import fake_run, fake_settings
14 | from vero_status.monitor import Monitor, perform_check
15 | from vero_status.server import create
16 |
17 |
18 | def free_port():
19 |     with socket.socket() as s:
20 |         s.bind(("127.0.0.1", 0))
21 |         return s.getsockname()[1]
22 |
23 |
24 | class PerformCheckTest(unittest.TestCase):
25 |     def setUp(self):
26 |         self.tmp = tempfile.TemporaryDirectory()
27 |         self.work = Path(self.tmp.name) / "sandbox"
28 |         self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
29 |         self.env.start()
30 |
31 |     def tearDown(self):
32 |         self.env.stop()
33 |         self.tmp.cleanup()
34 |
35 |     def check(self, mode="ok", **settings):
36 |         os.environ["FAKE_VERO_MODE"] = mode
37 |         return perform_check(fake_settings(**settings), self.work, fake_run)
38 |
39 |     def test_available_with_both_models(self):
40 |         r = self.check()
41 |         self.assertTrue(r["available"], r["reason"])
42 |         self.assertEqual(r["asked_model"], "us.anthropic.claude-haiku-4-5-20251001-v1:0")_# pinned cheap model
43 |         self.assertEqual(r["configured"]["model"], "us.anthropic.claude-opus-5")_# Vero's own setting
44 |         self.assertEqual(r["vero_version"], "vero 2.3.3 (fake0000)")
45 |         self.assertEqual(r["reason"], "")
46 |         self.assertIsNotNone(r["seconds"])
47 |
48 |     def test_empty_model_uses_veros_own(self):
49 |         self.assertEqual(self.check(check_model="")["asked_model"], "us.anthropic.claude-opus-5")
50 |
51 |     def test_failures_are_not_available_with_reason(self):
52 |         down = self.check("down")
53 |         self.assertEqual(down["reason"], "Vero reported an error (exit code 1): provider unavailable (connect ETIMEDOUT)")
54 |         self.assertIsNone(down["seconds"])_# no answer, no response time
55 |         r = self.check("auth")
56 |         self.assertFalse(r["available"])
57 |         self.assertNotIn("abc123secret", r["reason"])
58 |         self.assertEqual(self.check("nocompletion")["reason"], "Vero finished without an answer")
59 |
60 |     def test_refusal_is_available_with_note(self):
61 |         r = self.check("refuse")
62 |         self.assertTrue(r["available"])
63 |         self.assertIn("not with OK", r["reason"])
64 |
65 |     def test_vero_missing(self):
66 |         with mock.patch("vero_status.monitor.vero.find_vero", return_value=None):

```

```

67 |         r = perform_check(fake_settings(), self.work, lambda *a: self.fail("must not run"))
68 |         self.assertFalse(r["available"])
69 |         self.assertTrue(r["reason"].startswith("Vero CLI not found"))
70 |
71 |     def test_hang_times_out(self):
72 |         os.environ["FAKE_VERO_MODE"] = "hang"
73 |         s = fake_settings(timeout_seconds=30)
74 |         t0 = time.time()
75 |         r = perform_check(s, self.work, lambda a, t, c=None: fake_run(a, min(t, 1.0), c))_# shorten the wait
76 |         self.assertFalse(r["available"])
77 |         self.assertIn("no answer within", r["reason"])
78 |         self.assertLess(time.time() - t0, 6)
79 |
80 |     def test_secret_prompt_never_in_record(self):
81 |         self.assertNotIn("FAKE-SECRET-PROMPT", json.dumps(self.check()))
82 |
83 |
84 | class MonitorTest(unittest.TestCase):
85 |     def setUp(self):
86 |         self.tmp = tempfile.TemporaryDirectory()
87 |         self.dir = Path(self.tmp.name)
88 |         self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
89 |         self.env.start()
90 |
91 |     def tearDown(self):
92 |         self.env.stop()
93 |         self.tmp.cleanup()
94 |
95 |     def wait_idle(self, m):
96 |         for _ in range(100):
97 |             if not m.checking:
98 |                 return
99 |             time.sleep(0.05)
100 |         self.fail("check did not finish")
101 |
102 |     def make(self):
103 |         m = Monitor(self.dir, fake_run)
104 |         m.settings = fake_settings()
105 |         return m
106 |
107 |     def test_check_history_state_and_persistence(self):
108 |         m = self.make()
109 |         self.assertTrue(m.check_now())
110 |         self.assertFalse(m.check_now())_# one at a time
111 |         self.wait_idle(m)
112 |         st = m.state()
113 |         self.assertTrue(st["last"]["available"])
114 |         self.assertEqual(st["availability_24h"], st["checks_24h"], (100.0, 1))
115 |         self.assertEqual(st["configured"]["model"], "us.anthropic.claude-opus-5")
116 |         self.assertIsNotNone(st["next_check"])
117 |         self.assertEqual(len(Monitor(self.dir, fake_run).history), 1)_# saved to disk
118 |
119 |     def test_since_tracks_the_current_streak(self):
120 |         m = self.make()
121 |         m.history = [{"time": 10, "available": True}, {"time": 20, "available": False}, {"time": 30, "available": False}]
122 |         self.assertEqual(m.state()["since"], 20)
123 |
124 |     def test_settings_update_and_interval_off(self):
125 |         m = self.make()
126 |         m.update_settings({"interval_minutes": 0})
127 |         self.assertIsNone(m.next_check)
128 |         self.assertEqual(json.loads((self.dir / "settings.json").read_text())["interval_minutes"], 0)
129 |
130 |     def test_scheduler_runs_when_due(self):
131 |         m = self.make()
132 |         m.start(check_at_start=False)
133 |         m.next_check = time.time() + 0.2

```

```

134 |         m._wake.set()
135 |         for _ in range(100):
136 |             if m.history:
137 |                 break
138 |             time.sleep(0.05)
139 |         m.stop()
140 |         self.assertEqual(m.history[-1]["trigger"], "auto")
141 |
142 |
143 | class ServerTest(unittest.TestCase):
144 |     def setUp(self):
145 |         self.tmp = tempfile.TemporaryDirectory()
146 |         self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
147 |         self.env.start()
148 |         self.port = free_port()
149 |         self.mon = Monitor(Path(self.tmp.name), fake_run)
150 |         self.mon.settings = fake_settings()
151 |         self.quit = threading.Event()
152 |         self.srv = create(self.mon, self.port, self.quit.set)
153 |         threading.Thread(target=self.srv.serve_forever, daemon=True).start()
154 |         self.base = f"http://127.0.0.1:{self.port}"
155 |
156 |     def tearDown(self):
157 |         self.env.stop()
158 |         if not self.quit.is_set():
159 |             self.srv.shutdown()
160 |         self.srv.server_close()
161 |         self.tmp.cleanup()
162 |
163 |     def req(self, path, data=None, **headers):
164 |         body = json.dumps(data).encode() if data is not None else (b"" if headers.pop("post", False) else None)
165 |         r = urllib.request.Request(self.base + path, data=body, headers={"Content-Type": "application/json", **headers})
166 |         try:
167 |             with urllib.request.urlopen(r) as resp:
168 |                 return resp.status, json.loads(resp.read() or b"null")
169 |         except urllib.error.HTTPError as e:
170 |             return e.code, json.loads(e.read() or b"null")
171 |
172 |     def test_page_state_health(self):
173 |         with urllib.request.urlopen(self.base + "/") as r:
174 |             self.assertIn(b"Vero Status", r.read())
175 |             code, st = self.req("/api/state")
176 |             self.assertEqual((code, st["last"]), (200, None))
177 |             self.assertEqual(self.req("/api/health")[1]["app"], "vero-status")
178 |             self.assertEqual(self.req("/nope")[0], 404)
179 |
180 |     def test_check_now(self):
181 |         self.assertEqual(self.req("/api/check", post=True)[0], 202)
182 |         for _ in range(100):
183 |             if self.mon.history and not self.mon.checking:
184 |                 break
185 |             time.sleep(0.05)
186 |         self.assertTrue(self.req("/api/state")[1]["last"]["available"])
187 |
188 |     def test_settings_endpoint(self):
189 |         self.assertEqual(self.req("/api/settings", {"interval_minutes": 30})[0], 200)
190 |         self.assertEqual(self.mon.settings.interval_minutes, 30)
191 |         code, body = self.req("/api/settings", {"interval_minutes": 7})
192 |         self.assertEqual(code, 400)
193 |         self.assertIn("interval", body["error"])
194 |         self.assertEqual(self.req("/api/settings", [1])[0], 400)
195 |
196 |     def test_foreign_host_and_origin_rejected(self):
197 |         self.assertEqual(self.req("/api/state", Host="evil.example")[0], 403)
198 |         self.assertEqual(self.req("/api/check", post=True, Origin="http://evil.example")[0], 403)
199 |         self.assertEqual(self.req("/api/quit", post=True, Origin="http://evil.example")[0], 403)
200 |

```

```
201 |     def test_quit(self):
202 |         self.assertEqual(self.req("/api/quit", post=True)[1], {"stopping": True})
203 |         self.assertTrue(self.quit.wait(5))
204 |
205 |
206 | if __name__ == "__main__":
207 |     unittest.main()
=== END FILE: tests/test_monitor_server.py ===
```

```
=== FILE: tests/test_settings.py (45 lines) ===
```

```

1 | import json
2 | import tempfile
3 | import unittest
4 | from pathlib import Path
5 |
6 | import tests.helpers_ as noqa: F401
7 | from vero_status.settings import CHEAP_MODEL, Settings, SettingsError, load, save
8 |
9 |
10 | class SettingsTest(unittest.TestCase):
11 |     def test_defaults(self):
12 |         s = Settings()
13 |         s.validate()
14 |         self.assertEqual((s.check_model, s.interval_minutes, s.port), (CHEAP_MODEL, 15, 8767))
15 |
16 |     def test_merge_validates(self):
17 |         self.assertEqual(Settings().merged({"interval_minutes": 0, "check_model": ""}).interval_minutes, 0)
18 |         for bad in (
19 |             {"interval_minutes": 7},
20 |             {"interval_minutes": "15"},
21 |             {"timeout_seconds": 5},
22 |             {"port": 80},
23 |             {"check_model": "two words"},
24 |             {"evil": 1},
25 |             {"vero_path": 3},
26 |         ):
27 |             with self.assertRaises(SettingsError, msg=bad):
28 |                 Settings().merged(bad)
29 |
30 |     def test_load_save_and_broken_file(self):
31 |         with tempfile.TemporaryDirectory() as d:
32 |             p = Path(d) / "settings.json"
33 |             self.assertEqual(load(p), Settings())
34 |             save(p, Settings().merged({"interval_minutes": 30}))
35 |             self.assertEqual(load(p).interval_minutes, 30)
36 |             p.write_bytes(b"\xef\xbb\xbf" + json.dumps({"interval_minutes": 60, "old_key": 1}).encode())
37 |             self.assertEqual(load(p).interval_minutes, 60) # BOM and unknown keys tolerated
38 |             p.write_text("{broken}")
39 |             self.assertEqual(load(p), Settings())
40 |             p.write_text('{"interval_minutes": 7}')
41 |             self.assertEqual(load(p), Settings())
42 |
43 |
44 | if __name__ == "__main__":
45 |     unittest.main()
=== END FILE: tests/test_settings.py ===
```

=== FILE: tests/test_vero.py (98 lines) ===

```

1 | import os
2 | import sys
3 | import tempfile
4 | import time
5 | import unittest
6 | from pathlib import Path
7 | from unittest import mock
8 |
9 | import tests.helpers_ as noqa: F401
10 | from vero_status import vero
11 |
12 | GUIDE_STREAM = [
13 |     {'type': "task_started", "taskId": "1790868292727"},
14 |     {'type': "say", "say": "task", "text": "Reply with exactly: OK", "modelInfo": {"providerId": "bedrock",
15 |     'modelId': "us.anthropic.claude-haiku-4-5-20251001-v1:0", "mode": "act"}},
16 |     {'type': "say", "say": "api_req_started", "text": "\\request\\": \\secret prompt\\\"}},
17 |     {'type': "say", "say": "text", "text": "0", "partial": true},
18 |     {'type': "say", "say": "error", "text": "[ERROR] You did not use a tool in your previous response!"},
19 |     {'type': "say", "say": "completion_result", "text": "OK"}
20 | ]
21 |
22 |
23 | class ParseTaskStreamTest(unittest.TestCase):
24 |     def test_stream_from_integration_guide(self):
25 |         s = vero.parse_task_stream(GUIDE_STREAM)
26 |         self.assertEqual((s.answer, s.provider, s.model), ("OK", "bedrock", "us.anthropic.claude-haiku-4-5-20251001-v1:0"))
27 |         self.assertEqual(s.events, 6)
28 |
29 |     def test_only_completion_result_counts_andpartials_ignored(self):
30 |         s = vero.parse_task_stream(['say': "text", "text": "OK"}, {'say': "completion_result", "text": "0", "partial": true}])
31 |         self.assertIsNone(s.answer)
32 |
33 |     def test_garbage_ignored(self):
34 |         s = vero.parse_task_stream(["", "banner", "{bad", "[1]", '{"type": "error", "message": "Bearer abc.def failed"}'])
35 |         self.assertEqual((s.events, s.error), (1, "[hidden] failed"))
36 |
37 |
38 | class ParseConfigTest(unittest.TestCase):
39 |     def test_key_value_lines_as_in_the_guide(self):
40 |         text = (
41 |             "actModeApiProvider : bedrock\n"
42 |             "actModeApiModelId: us.anthropic.claude-opus-5\n"
43 |             "awsRegion: us-west-2\n"
44 |             "other : x"
45 |         )
46 |         self.assertEqual(
47 |             vero.parse_config(text), {"provider": "bedrock", "model": "us.anthropic.claude-opus-5", "region": "us-west-2"}
48 |         )
49 |
50 |     def test_json_and_quoted_variants(self):
51 |         self.assertEqual(vero.parse_config({'actModeApiModelId': "m1", "awsRegion": "eu"}), {"model": "m1", "region": "eu"})
52 |         self.assertEqual(vero.parse_config(' "actModeApiModelId": "m2", ', {"model": "m2"})
53 |         self.assertEqual(vero.parse_config("nothing useful"), {})
54 |
55 |
56 | class SmallParsersTest(unittest.TestCase):
57 |     def test_version_and_clean(self):
58 |         self.assertEqual(vero.parse_version("vero 2.3.3 (35b52374)\nmore"), "vero 2.3.3 (35b52374)")
59 |         self.assertEqual(vero.parse_version(""), "")
60 |         self.assertNotIn("abc123", vero.clean("error token=abc123 password=x"))
61 |         self.assertEqual(len(vero.clean("x" * 999)), vero.DETAIL_MAX)
62 |
63 |     def test_task_args(self):
64 |         a = vero.task_args("vero", "", 120, Path("w"))
65 |         self.assertEqual(a, ["vero", "task", "--json", "-t", "120", "-c", "w", vero.PROMPT])
66 |         self.assertIn("-m", vero.task_args("vero", "m1", 120, Path("w")))

```

```

67 |         self.assertNotIn("--yolo", a)
68 |
69 |
70 |     class RunAndFindTest(unittest.TestCase):
71 |     def test_run_collects_output(self):
72 |         r = vero.run([sys.executable, "-c", "print('hi'); import sys; sys.exit(3)"], 10)
73 |         self.assertEqual((r.started, r.code, r.out.strip(), r.timed_out), (True, 3, "hi", False))
74 |
75 |     def test_run_missing_program(self):
76 |         self.assertFalse(vero.run(["no-such-program-xyz"], 5).started)
77 |
78 |     @unittest.skipIf(os.name == "nt", "POSIX process group")
79 |     def test_timeout_kills_the_whole_tree_fast(self):
80 |         t0 = time.time()
81 |         r = vero.run(["sh", "-c", "sleep 30 & sleep 30; echo late"], 0.5)
82 |         self.assertTrue(r.timed_out)
83 |         self.assertLess(time.time() - t0, 3)
84 |
85 |     def test_find_vero(self):
86 |         self.assertEqual(vero.find_vero(sys.executable), sys.executable)
87 |         with mock.patch("vero_status.vero.shutil.which", return_value=None), mock.patch.dict(os.environ, {"APPDATA": ""}):
88 |             self.assertIsNone(vero.find_vero("/nope/vero-xyz"))
89 |             with tempfile.TemporaryDirectory() as d:
90 |                 npm = Path(d) / "npm"
91 |                 npm.mkdir()
92 |                 (npm / "vero.cmd").write_text("@echo off")
93 |                 with mock.patch("vero_status.vero.shutil.which", return_value=None), mock.patch.dict(os.environ, {"APPDATA": d}):
94 |                     self.assertEqual(vero.find_vero(""), str(npm / "vero.cmd"))_# npm default location on Windows
95 |
96 |
97 | if __name__ == "__main__":
98 |     unittest.main()
=== END FILE: tests/test_vero.py ===

```

```
=== FILE: vero_status/__init__.py (4 lines) ===
 1 | """Vero Status: a small dashboard that shows whether Vero is available and which model it uses."""
 2 |
 3 | __version__ = "4.0.0"
 4 | APP_ID = "vero-status"
=== END FILE: vero_status/__init__.py ===
```

```
=== FILE: vero_status/app.py (84 lines) ===
```

```

1 | """Program start: one instance, dashboard in the browser, no console window."""
2 |
3 | from __future__ import annotations
4 |
5 | import json
6 | import logging
7 | import logging.handlers
8 | import sys
9 | import threading
10 | import urllib.request
11 | import webbrowser
12 | from pathlib import Path
13 |
14 | from . import APP_ID
15 | from .monitor import Monitor
16 | from .server import create
17 |
18 | ROOT = Path(__file__).resolve().parents[1]
19 | DATA = ROOT / "data"
20 | log = logging.getLogger("vero_status")
21 |
22 |
23 | def message(text: str) -> None:
24 |     """Tell the user something without a console (pythonw): a Windows message box."""
25 |     if sys.platform == "win32":
26 |         import ctypes
27 |
28 |         ctypes.windll.user32.MessageBoxW(None, text, "Vero Status", 0x40)
29 |     else:
30 |         print(text)
31 |
32 |
33 | def already_running(port: int) -> bool:
34 |     try:
35 |         with urllib.request.urlopen(f"http://127.0.0.1:{port}/api/health", timeout=2) as r:
36 |             return bool(json.load(r).get("app") == APP_ID)
37 |     except Exception:
38 |         return False
39 |
40 |
41 | def setup_logging() -> None:
42 |     DATA.mkdir(parents=True, exist_ok=True)
43 |     handler = logging.handlers.RotatingFileHandler(DATA / "vero-status.log", maxBytes=500_000, backupCount=2, encoding="utf-8")
44 |     logging.basicConfig(level=logging.INFO, handlers=[handler], format="%(asctime)s %(levelname)s %(message)s", force=True)
45 |
46 |
47 | def main() -> int:
48 |     setup_logging()
49 |     monitor = Monitor(DATA)
50 |     port = monitor.settings.port
51 |     url = f"http://127.0.0.1:{port}/"
52 |     if already_running(port):
53 |         webbrowser.open(url) # second double-click: just show the dashboard
54 |         return 0
55 |     try:
56 |         srv = create(monitor, port)
57 |     except OSError as e:
58 |         log.error("cannot use port %s: %s", port, e)
59 |         message(
60 |             f"Vero Status cannot start: port {port} is used by another program.\n\n"
61 |             f'Change "port" in {DATA / "settings.json"} and start it again.'
62 |         )
63 |         return 1
64 |     monitor.start(check_at_start=True)
65 |     log.info("dashboard %s", url)
66 |     threading.Timer(0.3, webbrowser.open, args=(url,)).start()

```

```
67 |     try:
68 |         srv.serve_forever()
69 |     except KeyboardInterrupt:
70 |         pass
71 |     finally:
72 |         monitor.stop()
73 |         srv.server_close()
74 |         log.info("stopped")
75 |     return 0
76 |
77 |
78 | def run() -> None:
79 |     try:
80 |         sys.exit(main())
81 |     except Exception as e: # last resort: never die silently
82 |         logging.getLogger("vero_status").exception("crash")
83 |         message(f"Vero Status stopped because of an error:\n{e}\n\nDetails: {DATA / 'vero-status.log'}")
84 |         sys.exit(1)
=== END FILE: vero_status/app.py ===
```

```
=== FILE: vero_status/monitor.py (198 lines) ===
```

```

1 | """The monitor: runs a check on a timer or on demand, keeps the latest result and a short history."""
2 |
3 | from __future__ import annotations
4 |
5 | import json
6 | import logging
7 | import threading
8 | import time
9 | from pathlib import Path
10 | from typing import Any, Callable, Dict, List, Optional
11 |
12 | from . import vero
13 | from .settings import Settings, load, save
14 |
15 | log = logging.getLogger("vero_status")
16 | HISTORY_MAX = 500
17 | Runner = Callable[[List[str], float, Optional[str]], vero.Proc]
18 |
19 |
20 | def perform_check(settings: Settings, workdir: Path, run: Runner = vero.run, now: Optional[float] = None) -> Dict[str, Any]:
21 |     """One availability check. Returns a JSON-ready record; never raises for Vero problems."""
22 |     rec: Dict[str, Any] = {
23 |         "time": time.time() if now is None else now,
24 |         "available": False,
25 |         "reason": "",
26 |         "seconds": None,
27 |         "asked_model": None,
28 |         "provider": None,
29 |         "vero_version": None,
30 |         "configured": {},
31 |     }
32 |     exe = vero.find_vero(settings.vero_path)
33 |     if not exe:
34 |         rec["reason"] = "Vero CLI not found on this computer (set its path in Settings)"
35 |         return rec
36 |     ver = run([exe, "version"], 30, None)
37 |     if not ver.started or ver.timed_out or ver.code != 0:
38 |         rec["reason"] = "Vero CLI does not start: " + vero.clean(ver.err or ver.out or "no output")
39 |         return rec
40 |     rec["vero_version"] = vero.parse_version(ver.out)
41 |     cfg = run([exe, "config"], 30, None)
42 |     if cfg.started and cfg.code == 0:
43 |         rec["configured"] = vero.parse_config(cfg.out)
44 |     workdir.mkdir(parents=True, exist_ok=True) # empty folder: the agent has nothing to read or change
45 |     limit = settings.timeout_seconds
46 |     task = run(vero.task_args(exe, settings.check_model, limit, workdir), limit + 30, str(workdir))
47 |     stream = vero.parse_task_stream(task.out.splitlines())
48 |     rec["asked_model"], rec["provider"] = stream.model, stream.provider # model Vero used for the check
49 |     if task.timed_out:
50 |         rec["reason"] = f"no answer within {limit + 30} s"
51 |     elif stream.answer is not None:
52 |         rec["available"] = True
53 |         rec["seconds"] = round(task.seconds, 1) # only a real answer has a response time
54 |         if "ok" not in stream.answer.lower():
55 |             rec["reason"] = "answered, but not with OK (a guardrail or the model replied differently)"
56 |     elif stream.error:
57 |         rec["reason"] = stream.error
58 |     elif task.code:
59 |         msg = vero.clean(task.err or task.out or "no output")
60 |         msg = msg[6:].lstrip() if msg.lower().startswith("error:") else msg
61 |         rec["reason"] = f"Vero reported an error (exit code {task.code}): {msg}"
62 |     elif stream.events == 0:
63 |         rec["reason"] = "Vero returned no JSON events (is this Vero CLI 2.3 or newer?)"
64 |     else:
65 |         rec["reason"] = "Vero finished without an answer"
66 |     return rec

```

```

67 |
68 |
69 | class Monitor:
70 |     def __init__(self, data_dir: Path, run: Runner = vero.run):
71 |         self.data_dir = data_dir
72 |         self.run = run
73 |         self.settings_path = data_dir / "settings.json"
74 |         self.history_path = data_dir / "history.json"
75 |         self.settings = load(self.settings_path)
76 |         self.history: List[Dict[str, Any]] = self._load_history()
77 |         self.checking = False
78 |         self.next_check: Optional[float] = None
79 |         self._lock = threading.Lock()
80 |         self._wake = threading.Event()
81 |         self._stop = threading.Event()
82 |
83 |     # history -----
84 |     def _load_history(self) -> List[Dict[str, Any]]:
85 |         try:
86 |             data = json.loads(self.history_path.read_text(encoding="utf-8"))
87 |             return [r for r in data if isinstance(r, dict) and "time" in r][-HISTORY_MAX:]
88 |         except (OSError, ValueError):
89 |             return []
90 |
91 |     def _save_history(self) -> None:
92 |         self.data_dir.mkdir(parents=True, exist_ok=True)
93 |         tmp = self.history_path.with_suffix(".tmp")
94 |         tmp.write_text(json.dumps(self.history[-HISTORY_MAX:]), encoding="utf-8")
95 |         tmp.replace(self.history_path)
96 |
97 |     # checks -----
98 |     def check_now(self, trigger: str = "manual") -> bool:
99 |         """Start a check in the background. False if one is already running."""
100 |         with self._lock:
101 |             if self.checking:
102 |                 return False
103 |             self.checking = True
104 |             threading.Thread(target=self._check, args=(trigger,), daemon=True).start()
105 |             return True
106 |
107 |     def _check(self, trigger: str) -> None:
108 |         try:
109 |             rec = perform_check(self.settings, self.data_dir / "sandbox", self.run)
110 |             except Exception as e: # a bug must never kill the program
111 |                 log.exception("check failed")
112 |             rec = {
113 |                 "time": time.time(),
114 |                 "available": False,
115 |                 "reason": f"monitor error: {type(e).__name__}",
116 |                 "seconds": None,
117 |                 "asked_model": None,
118 |                 "provider": None,
119 |                 "vero_version": None,
120 |                 "configured": {},
121 |             }
122 |             rec["trigger"] = trigger
123 |             with self._lock:
124 |                 self.history.append(rec)
125 |                 self.history = self.history[-HISTORY_MAX:]
126 |                 self._save_history()
127 |                 self.checking = False
128 |                 log.info("check: %s %s", "AVAILABLE" if rec["available"] else "NOT AVAILABLE", rec["reason"])
129 |                 self._plan_next()
130 |
131 |     def _plan_next(self) -> None:
132 |         m = self.settings.interval_minutes
133 |         self.next_check = time.time() + m * 60 if m else None

```

```

134 |         self._wake.set()
135 |
136 |     def start(self, check_at_start: bool = True) -> None:
137 |         def loop() -> None:
138 |             while not self._stop.is_set():
139 |                 self._wake.clear()
140 |                 due = self.next_check
141 |                 wait = None if due is None else max(0.0, due - time.time())
142 |                 if self._wake.wait(wait) or self._stop.is_set():
143 |                     continue # plan changed: re-evaluate
144 |                 if due is not None and time.time() >= due:
145 |                     self.next_check = None
146 |                     if not self.check_now("auto"):
147 |                         self._plan_next()
148 |
149 |             threading.Thread(target=loop, daemon=True).start()
150 |             if check_at_start:
151 |                 self.check_now("start")
152 |             else:
153 |                 self._plan_next()
154 |
155 |     def stop(self) -> None:
156 |         self._stop.set()
157 |         self._wake.set()
158 |
159 |     # settings -----
160 |     def update_settings(self, changes: Dict[str, Any]) -> Settings:
161 |         new = self.settings.merged(changes)
162 |         save(self.settings_path, new)
163 |         self.settings = new
164 |         if not self.checking:
165 |             self._plan_next()
166 |         return new
167 |
168 |     # view -----
169 |     def state(self) -> Dict[str, Any]:
170 |         with self._lock:
171 |             hist = list(self.history)
172 |             day = [r for r in hist if r["time"] >= time.time() - 86400]
173 |             last = hist[-1] if hist else None
174 |             last_ok = next((r for r in reversed(hist) if r.get("available")), None)
175 |             since = None
176 |             if last:
177 |                 since = last["time"]
178 |                 for r in reversed(hist):
179 |                     if bool(r.get("available")) != bool(last.get("available")):
180 |                         break
181 |                 since = r["time"]
182 |             return {
183 |                 "checking": self.checking,
184 |                 "next_check": self.next_check,
185 |                 "last": last,
186 |                 "since": since,
187 |                 "configured": (last or {}).get("configured") or (last_ok or {}).get("configured") or {},
188 |                 "availability_24h": round(sum(1 for r in day if r.get("available")) / len(day) * 100, 1) if day else None,
189 |                 "checks_24h": len(day),
190 |                 "history": hist[-48:],
191 |                 "settings": {
192 |                     "vero_path": self.settings.vero_path,
193 |                     "vero_found": vero.find_vero(self.settings.vero_path),
194 |                     "check_model": self.settings.check_model,
195 |                     "interval_minutes": self.settings.interval_minutes,
196 |                     "timeout_seconds": self.settings.timeout_seconds,
197 |                 },
198 |             }
=== END FILE: vero_status/monitor.py ===

```

```

=== FILE: vero_status/server.py (104 lines) ===
1 | """Local web server for the dashboard. Binds to 127.0.0.1 only.
2 | Host and Origin are checked so other web pages cannot use it."""
3 |
4 | from __future__ import annotations
5 |
6 | import json
7 | import logging
8 | import threading
9 | from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
10 | from pathlib import Path
11 | from typing import Any, Callable, Dict, Optional, Type
12 | from urllib.parse import urlparse
13 |
14 | from . import APP_ID, __version__
15 | from .monitor import Monitor
16 | from .settings import SettingsError
17 |
18 | log = logging.getLogger("vero_status")
19 | INDEX = Path(__file__).parent / "web" / "index.html"
20 | MAX_BODY = 10_000
21 |
22 |
23 | def make_handler(monitor: Monitor, port: int, on_quit: Callable[[], None]) -> Type[BaseHTTPRequestHandler]:
24 |     """hosts = {"127.0.0.1":{port}", "localhost:{port}"}
25 |
26 |     """class Handler(BaseHTTPRequestHandler):
27 |         """server_version = f"{APP_ID}/{__version__}"
28 |
29 |         """def log_message(self, format: str, *args: Any) -> None:
30 |             """log.debug("%s " + format, self.address_string(), *args)
31 |
32 |             """def _send(self, code: int, body: bytes, ctype: str) -> None:
33 |                 """self.send_response(code)
34 |                 """self.send_header("Content-Type", ctype)
35 |                 """self.send_header("Content-Length", str(len(body)))
36 |                 """self.send_header("Cache-Control", "no-store")
37 |                 """self.send_header("X-Content-Type-Options", "nosniff")
38 |                 """self.end_headers()
39 |                 """self.wfile.write(body)
40 |
41 |             """def _json(self, obj: Any, code: int = 200) -> None:
42 |                 """self._send(code, json.dumps(obj).encode(), "application/json")
43 |
44 |             """def _allowed(self, post: bool) -> bool:
45 |                 """host = (self.headers.get("Host") or "").lower()
46 |                 """if host not in hosts:
47 |                     """return False
48 |                 """origin = self.headers.get("Origin")
49 |                 """return not post or origin is None or urlparse(origin).netloc.lower() == host
50 |
51 |             """def do_GET(self) -> None:
52 |                 """if not self._allowed(post=False):
53 |                     """return self._json({"error": "forbidden"}, 403)
54 |                 """path = urlparse(self.path).path
55 |                 """if path in ("/", "/index.html"):
56 |                     """self._send(200, INDEX.read_bytes(), "text/html; charset=utf-8")
57 |                 """elif path == "/api/state":
58 |                     """self._json({"version": __version__, **monitor.state()})
59 |                 """elif path == "/api/health":
60 |                     """self._json({"app": APP_ID, "version": __version__})
61 |                 """else:
62 |                     """self._json({"error": "not found"}, 404)
63 |
64 |             """def do_POST(self) -> None:
65 |                 """if not self._allowed(post=True):
66 |                     """return self._json({"error": "forbidden"}, 403)

```

```

67 | """path = urlparse(self.path).path
68 | """if path == "/api/check":
69 | """started = monitor.check_now("manual")
70 | """return self._json({"started": started}, 202 if started else 409)
71 | """if path == "/api/settings":
72 | """try:
73 | """length = int(self.headers.get("Content-Length") or 0)
74 | """if not 0 < length <= MAX_BODY:
75 | """raise SettingsError("empty or too large request")
76 | """changes = json.loads(self.rfile.read(length))
77 | """if not isinstance(changes, dict):
78 | """raise SettingsError("expected a JSON object")
79 | """monitor.update_settings(changes)
80 | """except (SettingsError, ValueError, TypeError) as e:
81 | """return self._json({"error": str(e)}, 400)
82 | """return self._json({"saved": True})
83 | """if path == "/api/quit":
84 | """self._json({"stopping": True})
85 | """threading.Thread(target=on_quit, daemon=True).start()
86 | """return None
87 | """return self._json({"error": "not found"}, 404)
88 |
89 | """return Handler
90 |
91 |
92 | def create(monitor: Monitor, port: int, on_quit: Optional[Callable[[], None]] = None) -> ThreadingHTTPServer:
93 | """holder: Dict[str, ThreadingHTTPServer] = {}
94 |
95 | """def quit_() -> None:
96 | """monitor.stop()
97 | """holder["srv"].shutdown()
98 | """if on_quit:
99 | """on_quit()
100 |
101 | """srv = ThreadingHTTPServer(("127.0.0.1", port), make_handler(monitor, port, quit_))
102 | """srv.daemon_threads = True
103 | """holder["srv"] = srv
104 | """return srv
=== END FILE: vero_status/server.py ===

```

```
=== FILE: vero_status/settings.py (62 lines) ===
```

```

1 | """User settings, changed from the dashboard and kept in data/settings.json."""
2 |
3 | from __future__ import annotations
4 |
5 | import json
6 | from dataclasses import asdict, dataclass, replace
7 | from pathlib import Path
8 | from typing import Any, Dict
9 |
10 | INTERVALS = (0, 5, 15, 30, 60) # minutes; 0 = only when "Check now" is clicked
11 | CHEAP_MODEL = "us.anthropic.claude-haiku-4-5-20251001-v1:0"
12 |
13 |
14 | class SettingsError(ValueError):
15 |     """pass
16 |
17 |
18 | @dataclass(frozen=True)
19 | class Settings:
20 |     """vero_path: str = "" # empty = find it automatically
21 |     """check_model: str = CHEAP_MODEL # empty = Vero's own configured model
22 |     """interval_minutes: int = 15
23 |     """timeout_seconds: int = 120
24 |     """port: int = 8767
25 |
26 |     """def merged(self, changes: Dict[str, Any]) -> Settings:
27 |         """A validated copy with the given changes (unknown keys are rejected)."""
28 |         """allowed = set(asdict(self))
29 |         """unknown = set(changes) - allowed
30 |         """if unknown:
31 |             raise SettingsError(f"unknown setting(s): {' '.join(sorted(unknown))}")
32 |         """new = replace(self, **changes)
33 |         """new.validate()
34 |         """return new
35 |
36 |     """def validate(self) -> None:
37 |         """if not isinstance(self.vero_path, str) or not isinstance(self.check_model, str):
38 |             raise SettingsError("vero_path and check_model must be text")
39 |         """if len(self.check_model) > 200 or any(c.isspace() for c in self.check_model):
40 |             raise SettingsError("the model id must be one word without spaces")
41 |         """if self.interval_minutes not in INTERVALS:
42 |             raise SettingsError(f"interval must be one of {INTERVALS} minutes")
43 |         """if not isinstance(self.timeout_seconds, int) or not 30 <= self.timeout_seconds <= 600:
44 |             raise SettingsError("timeout must be between 30 and 600 seconds")
45 |         """if not isinstance(self.port, int) or not 1024 <= self.port <= 65535:
46 |             raise SettingsError("port must be between 1024 and 65535")
47 |
48 |
49 | def load(path: Path) -> Settings:
50 |     """Settings from the file, or defaults. A broken file never stops the program."""
51 |     """try:
52 |         """raw = json.loads(path.read_text(encoding="utf-8-sig"))
53 |         """return Settings().merged({k: v for k, v in raw.items() if k in asdict(Settings())})
54 |     """except (OSError, ValueError, TypeError, AttributeError):
55 |         """return Settings()
56 |
57 |
58 | def save(path: Path, settings: Settings) -> None:
59 |     """path.parent.mkdir(parents=True, exist_ok=True)
60 |     """tmp = path.with_suffix(".tmp")
61 |     """tmp.write_text(json.dumps(asdict(settings), indent=2) + "\n", encoding="utf-8")
62 |     """tmp.replace(path)

```

```
=== END FILE: vero_status/settings.py ===
```



```

67 |     subprocess.run(
68 |         ["taskkill", "/T", "/F", "/PID", str(proc.pid)], capture_output=True, creationflags=NO_WINDOW, check=False
69 |     )
70 |     else:
71 |         os.killpg(proc.pid, signal.SIGKILL)
72 |     with contextlib.suppress(OSError):
73 |         proc.kill()
74 |
75 |
76 | def run(args: Sequence[str], timeout_s: float, cwd: Optional[str] = None) -> Proc:
77 |     """Start a process without a window, collect output, kill the whole tree on timeout."""
78 |     t0 = time.perf_counter()
79 |     try:
80 |         proc = subprocess.Popen(
81 |             list(args),
82 |             stdin=subprocess.DEVNULL,
83 |             stdout=subprocess.PIPE,
84 |             stderr=subprocess.PIPE,
85 |             cwd=cwd,
86 |             creationflags=NO_WINDOW,
87 |             start_new_session=os.name != "nt",
88 |         )
89 |     except OSError as e:
90 |         return Proc(False, None, "", f"cannot start {Path(args[0]).name}: {e}", 0.0, False)
91 |     stdout, stderr = proc.stdout, proc.stderr
92 |     if stdout is None or stderr is None:
93 |         _kill_tree(proc)
94 |         return Proc(False, None, "", "no output pipes", 0.0, False)
95 |     out: List[bytes] = []
96 |     err: List[bytes] = []
97 |     readers = [
98 |         threading.Thread(target=lambda: out.append(stdout.read()), daemon=True),
99 |         threading.Thread(target=lambda: err.append(stderr.read()), daemon=True),
100 |     ]
101 |     for r in readers:
102 |         r.start()
103 |     timed_out = False
104 |     try:
105 |         proc.wait(timeout=timeout_s)
106 |     except subprocess.TimeoutExpired:
107 |         timed_out = True
108 |         _kill_tree(proc)
109 |         proc.wait()
110 |     seconds = time.perf_counter() - t0
111 |     for r in readers:
112 |         r.join(timeout=5)
113 |     stdout.close()
114 |     stderr.close()
115 |     return Proc(
116 |         True,
117 |         proc.returncode,
118 |         b"".join(out).decode("utf-8", "replace"),
119 |         b"".join(err).decode("utf-8", "replace"),
120 |         seconds,
121 |         timed_out,
122 |     )
123 |
124 |
125 | # ---- pure parsers -----
126 |
127 |
128 | def parse_version(text: str) -> str:
129 |     first = (text.strip().splitlines() or [""])[0]
130 |     return clean(first)[:60]
131 |
132 |
133 | CONFIG_KEYS = {

```

```

134 |     "actModeApiModelId": "model",
135 |     "actModeApiProvider": "provider",
136 |     "awsRegion": "region",
137 |     "planModeApiModelId": "plan_model",
138 | }
139 |
140 |
141 | def parse_config(text: str) -> Dict[str, str]:
142 |     """`vero config` output -> {model, provider, region, plan_model}. Accepts JSON or `key : value` lines."""
143 |     found: Dict[str, str] = {}
144 |     try:
145 |         data = json.loads(text)
146 |         if isinstance(data, dict):
147 |             for key, name in CONFIG_KEYS.items():
148 |                 if isinstance(data.get(key), (str, int, float)):
149 |                     found[name] = str(data[key])
150 |             return found
151 |     except json.JSONDecodeError:
152 |         pass
153 |         for line in text.splitlines():
154 |             m = re.match(r"^\s*['\"]?(\\w+)[\\\"]?\s*[:=]\s*['\"]?([^\",,]+)[\\\"]?\.?\s*$", line)
155 |             if m and m.group(1) in CONFIG_KEYS:
156 |                 found[CONFIG_KEYS[m.group(1)]] = m.group(2).strip()
157 |         return found
158 |
159 |
160 | @dataclass(frozen=True)
161 | class TaskStream:
162 |     answer: Optional[str]
163 |     provider: Optional[str]
164 |     model: Optional[str]
165 |     error: Optional[str]
166 |     events: int
167 |
168 |
169 | def parse_task_stream(lines: Iterable[str]) -> TaskStream:
170 |     answer = provider = model = error = None
171 |     events = 0
172 |     for raw in lines:
173 |         raw = raw.strip()
174 |         if not raw.startswith("{"):
175 |             continue
176 |         try:
177 |             ev = json.loads(raw)
178 |         except json.JSONDecodeError:
179 |             continue
180 |         if not isinstance(ev, dict):
181 |             continue
182 |         events += 1
183 |         info = ev.get("modelInfo")
184 |         if isinstance(info, dict):
185 |             provider = info.get("providerId") or provider
186 |             model = info.get("modelId") or model
187 |         if ev.get("partial") is True:
188 |             continue
189 |         if ev.get("say") == "completion_result":
190 |             answer = str(ev.get("text") or "")
191 |             elif ev.get("say") in ("error", "api_req_failed") or ev.get("type") == "error":
192 |                 error = clean(str(ev.get("text") or ev.get("message") or "error"))
193 |         return TaskStream(answer, provider, model, error, events)
194 |
195 |
196 | def task_args(exe: str, model: str, timeout_s: int, workdir: Path, prompt: str = PROMPT) -> List[str]:
197 |     args = [exe, "task", "--json", "-t", str(timeout_s), "-c", str(workdir)]
198 |     if model:
199 |         args += ["-m", model]
200 |     return [*args, prompt]  # never --yolo: the probe needs no file or shell access

```

=== END FILE: vero_status/vero.py ===

=== FILE: vero_status/web/index.html (250 lines) ===

```

1 | <!doctype html>
2 | <html lang="en">
3 | <head>
4 | <meta charset="utf-8">
5 | <meta name="viewport" content="width=device-width, initial-scale=1">
6 | <title>Vero Status</title>
7 | <link rel="icon" id="favicon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 16 16'%3E%3Ccircle cx='8' cy='8' r='7' fill='%238a94a3'/%3E%3C/svg%3E">
8 | <style>
9 | :root{
10 |   --ink:#1b2230; --mut:#5d6878; --line:#dde2ea; --paper:#f4f6f9; --card:#ffffff; --brand:#0a6aa1;
11 |   --ok:#1f7a45; --ok-bg:#e8f4ed; --bad:#9b1c2e; --bad-bg:#f8e7ea; --warn:#a8500b; --idle:#8a94a3; --idle-bg:#eef1f5;
12 |   --sans:"Segoe UI","IBM Plex Sans",Roboto,Helvetica,Arial,sans-serif; --mono:Consolas,"IBM Plex Mono",ui-monospace,monospace;
13 | }
14 | @media (prefers-color-scheme: dark){:root{
15 |   --ink:#e6eaf0; --mut:#9aa5b4; --line:#2c3441; --paper:#10141a; --card:#171c24; --brand:#4fa8dc;
16 |   --ok:#4cc38a; --ok-bg:#132a1f; --bad:#f06b7f; --bad-bg:#321a20; --warn:#e39a4b; --idle:#6f7a89; --idle-bg:#1c222b;
17 | }}
18 | *{box-sizing:border-box}
19 | body{margin:0;background:var(--paper);color:var(--ink);font:15px/1.5 var(--sans)}
20 | .wrap{max-width:880px;margin:0 auto;padding:22px 20px 40px}
21 | header{display:flex;align-items:center;gap:12px;flex-wrap:wrap;margin-bottom:16px}
22 | header h1{font-size:20px;margin:0;font-weight:700}
23 | header .ver{color:var(--mut);font-size:13px}
24 | header .sp{flex:1}
25 | button{font:inherit;font-size:14px;border:1px solid var(--line);background:var(--card);color:var(--ink);border-radius:8px;padding:8px 14px;cursor:pointer}
26 | button.primary{background:var(--brand);border-color:var(--brand);color:#fff;font-weight:600}
27 | button.disabled{opacity:.55;cursor:default}
28 | button:focus-visible,input:focus-visible,select:focus-visible{outline:2px solid var(--brand);outline-offset:2px}
29 | .card{background:var(--card);border:1px solid var(--line);border-radius:12px;padding:18px 20px;margin-bottom:14px}
30 | .hero{display:flex;gap:20px;align-items:center;padding:26px 24px;background:var(--idle-bg)}
31 | .hero.ok{background:var(--ok-bg);border-color:color-mix(in srgb,var(--ok) 35%,var(--line))}
32 | .hero.bad{background:var(--bad-bg);border-color:color-mix(in srgb,var(--bad) 35%,var(--line))}
33 | .lamp{width:72px;height:72px;border-radius:50%;flex:none;display:flex;align-items:center;justify-content:center;color:#fff;font-size:36px;font-weight:800;background:var(--idle)}
34 | .ok .lamp{background:var(--ok)} .bad .lamp{background:var(--bad)}
35 | .checking .lamp{animation:pulse 1.2s ease-in-out infinite}
36 | @keyframes pulse{50%{opacity:.45}}
37 | @media (prefers-reduced-motion:reduce){.checking .lamp{animation:none}}
38 | .hero h2{margin:0;font-size:28px;line-height:1.2}
39 | .hero .meta{color:var(--mut);margin-top:4px}
40 | .hero .why{margin-top:8px;font-size:14px}
41 | .ok .why{color:var(--warn)} .bad .why{color:var(--bad)}
42 | .label{font-size:12px;text-transform:uppercase;letter-spacing:.06em;color:var(--mut);margin-bottom:6px}
43 | .model{display:grid;grid-template-columns:1fr 1fr;gap:18px}
44 | .model .name{font-size:22px;font-weight:700;line-height:1.25}
45 | .model .id{font-family:var(--mono);font-size:12px;color:var(--mut);overflow-wrap:anywhere;margin-top:2px}
46 | .model .sub{font-size:13px;color:var(--mut);margin-top:4px}
47 | .facts{display:grid;grid-template-columns:repeat(3,1fr);gap:14px}
48 | .facts .v{font-size:22px;font-weight:700;font-variant-numeric:tabular-nums}
49 | .dots{display:flex;gap:4px;flex-wrap:wrap;margin-top:4px}
50 | .dots span{width:14px;height:14px;border-radius:4px;background:var(--idle)}
51 | .dots span.ok{background:var(--ok)} .dots span.bad{background:var(--bad)}
52 | .muted{color:var(--mut);font-size:13px}
53 | .settings{display:none}
54 | .settings.open{display:block}
55 | .form{display:grid;grid-template-columns:220px 1fr;gap:12px 16px;align-items:center}
56 | .form input,.form select{font:inherit;font-size:14px;padding:7px 10px;border:1px solid var(--line);border-radius:8px;background:var(--card);color:var(--ink);width:100%}
57 | .form .hint{grid-column:2;font-size:12.5px;color:var(--mut);margin-top:6px}
58 | .row{display:flex;gap:10px;align-items:center;margin-top:14px}
59 | .msg{font-size:13px}
60 | .msg.err{color:var(--bad)} .msg.ok{color:var(--ok)}
61 | .overlay{position:fixed;inset:0;background:var(--paper);display:none;align-items:center;justify-content:center;text-align:center;padding:20px}
62 | .overlay.show{display:flex}
63 | .overlay h2{margin:0 0 8px}
64 | @media (max-width:640px){.model,.facts{grid-template-columns:1fr}.form{grid-template-columns:1fr}.form .hint{grid-column:1}
65 |   .hero{flex-direction:column;align-items:flex-start}.hero h2{font-size:23px}}
66 | </style>

```

```

67 | </head>
68 | <body>
69 | <div class="wrap">
70 |   <header>
71 |     <h1>Vero Status</h1><span class="ver" id="ver"></span>
72 |     <span class="sp"></span>
73 |     <button class="primary" id="checkBtn" type="button">Check now</button>
74 |     <button id="setBtn" type="button" aria-expanded="false" aria-controls="settings">Settings</button>
75 |     <button id="quitBtn" type="button">Quit</button>
76 |   </header>
77 |
78 |   <section class="card hero" id="hero" aria-live="polite">
79 |     <div class="lamp" id="lamp">?</div>
80 |     <div>
81 |       <h2 id="title">Starting...</h2>
82 |       <div class="meta" id="meta"></div>
83 |       <div class="why" id="why"></div>
84 |     </div>
85 |   </section>
86 |
87 |   <section class="card">
88 |     <div class="model">
89 |       <div>
90 |         <div class="label">Vero's model</div>
91 |         <div class="name" id="cfgName"></div>
92 |         <div class="id" id="cfgId"></div>
93 |         <div class="sub" id="cfgSub"></div>
94 |       </div>
95 |       <div>
96 |         <div class="label" id="ansLabel">Answered the last check</div>
97 |         <div class="name" id="ansName"></div>
98 |         <div class="id" id="ansId"></div>
99 |         <div class="sub" id="ansSub"></div>
100 |       </div>
101 |     </div>
102 |   </section>
103 |
104 |   <section class="card">
105 |     <div class="facts">
106 |       <div><div class="label">Available, last 24 h</div><div class="v" id="avail"></div><div class="muted" id="availSub"></div></div>
107 |       <div><div class="label">Last answer took</div><div class="v" id="took"></div><div class="muted">for "Reply with exactly: OK"</div></div>
108 |       <div><div class="label">Next check</div><div class="v" id="next"></div><div class="muted" id="every"></div></div>
109 |     </div>
110 |     <div class="label" style="margin-top:16px">Last checks (oldest → newest)</div>
111 |     <div class="dots" id="dots"></div>
112 |   </section>
113 |
114 |   <section class="card settings" id="settings" aria-label="Settings">
115 |     <div class="label">Settings</div>
116 |     <div class="form">
117 |       <label for="sInterval">Check automatically</label>
118 |       <select id="sInterval">
119 |         <option value="0">Only when I click "Check now"</option>
120 |         <option value="5">Every 5 minutes</option>
121 |         <option value="15">Every 15 minutes</option>
122 |         <option value="30">Every 30 minutes</option>
123 |         <option value="60">Every hour</option>
124 |       </select>
125 |       <div class="hint">Each check asks Vero one tiny question (about 1 minute, 2 small model calls).</div>
126 |       <label for="sModel">Model used for the check</label>
127 |       <input id="sModel" spellcheck="false" placeholder="empty = Vero's own model">
128 |       <div class="hint">A cheap model keeps checks cheap. Leave empty to test the model Vero is set to.</div>
129 |       <label for="sTimeout">Give up after (seconds)</label>
130 |       <input id="sTimeout" type="number" min="30" max="600" step="10">
131 |       <label for="sPath">Vero CLI location</label>
132 |       <input id="sPath" spellcheck="false" placeholder="empty = find it automatically">
133 |       <div class="hint" id="found"></div>

```

```

134 | </div>
135 | <div class="row"><button class="primary" id="saveBtn" type="button">Save</button><span class="msg" id="saveMsg"></span></div>
136 | </section>
137 |
138 | <p class="muted" id="foot"></p>
139 | </div>
140 |
141 | <div class="overlay" id="stopped"><div><h2>Vero Status is stopped</h2>
142 | <p class="muted">Double-click <b>Vero Status</b> to start it again. You can close this tab.</p></div></div>
143 |
144 | <script>
145 | (function(){
146 | "use strict";
147 | const $ = (id) => document.getElementById(id);
148 | let S = null, timer = null, settingsLoaded = false, offline = false;
149 |
150 | function prettyModel(id){
151 |   if (!id) return null;
152 |   const m = /claude-(opus|sonnet|haiku|fable)-(\d+)?(?:-(\d{1,2}))?(?!\\d)/i.exec(id);
153 |   if (m) return "Claude " + m[1][0].toUpperCase() + m[1].slice(1).toLowerCase() + " " + m[2] + (m[3] ? "." + m[3] : "");
154 |   return id.split(/[/:]/).filter(Boolean).slice(-1)[0] || id;
155 | }
156 | function ago(t){ const s = Math.max(0, Math.round(Date.now()/1000 - t));
157 |   if (s < 60) return s + " s ago"; if (s < 3600) return Math.round(s/60) + " min ago";
158 |   if (s < 86400) return Math.round(s/3600) + " h ago"; return Math.round(s/86400) + " d ago"; }
159 | const clock = (t) => new Date(t*1000).toLocaleTimeString(undefined, {hour:"2-digit", minute:"2-digit"});
160 | const day = (t) => new Date(t*1000).toLocaleString(undefined, {month:"short", day:"numeric", hour:"2-digit", minute:"2-digit"});
161 | function setIcon(color){ $("favicon").href = "data:image/svg+xml," + encodeURIComponent(
162 |   `<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 16 16'><circle cx='8' cy='8' r='7' fill='${color}'/></svg>`); }
163 |
164 | function render(){
165 |   const s = S, last = s.last;
166 |   $("ver").textContent = "v" + s.version;
167 |   const hero = $("hero");
168 |   let cls = "", title, icon = "?", meta = "", why = "";
169 |   if (s.checking && !last) { title = "Checking Vero..."; meta = "The first check takes about a minute."; }
170 |   else if (!last) { title = "Not checked yet"; meta = 'Click "Check now"'; }
171 |   else {
172 |     cls = last.available ? "ok" : "bad"; icon = last.available ? "✓" : "x";
173 |     title = last.available ? "Vero is available" : "Vero is not available";
174 |     meta = `since ${day(s.since)} · last check ${ago(last.time)}`;
175 |     if (s.checking) meta = "Checking again now... · " + meta;
176 |     why = last.reason || "";
177 |   }
178 |   hero.className = "card hero " + cls + (s.checking ? " checking" : "");
179 |   $("lamp").textContent = icon; $("title").textContent = title; $("meta").textContent = meta; $("why").textContent = why;
180 |   document.title = (last ? (last.available ? "✓" : "x") : "") + "Vero Status";
181 |   setIcon(!last ? "#8a94a3" : last.available ? "#1f7a45" : "#9b1c2e");
182 |
183 |   const c = s.configured || {};
184 |   $("cfgName").textContent = prettyModel(c.model) || "unknown";
185 |   $("cfgId").textContent = c.model || "";
186 |   $("cfgSub").textContent = [c.provider && "provider " + c.provider, c.region].filter(Boolean).join(" · ") || "from vero config";
187 |   const asked = last && last.asked_model;
188 |   $("ansLabel").textContent = !last || last.available ? "Answered the last check" : "Asked in the last check: no answer";
189 |   $("ansName").textContent = prettyModel(asked) || "—";
190 |   $("ansId").textContent = asked || "";
191 |   $("ansSub").textContent = last ? [last.provider && "provider " + last.provider, last.vero_version].filter(Boolean).join(" · ") : "";
192 |
193 |   $("avail").textContent = s.availability_24h == null ? "—" : s.availability_24h + "%";
194 |   $("availSub").textContent = s.checks_24h ? `${s.checks_24h} check${s.checks_24h > 1 ? "s" : ""}` : "no checks yet";
195 |   $("took").textContent = last && last.available && last.seconds != null ? Math.round(last.seconds) + " s" : "—";
196 |   const iv = s.settings.interval_minutes;
197 |   $("every").textContent = iv ? `every ${iv} min` : "automatic checks are off";
198 |   tick();
199 |   $("dots").innerHTML = s.history.map(r => `<span class="${r.available ? "ok" : "bad"}" title="${day(r.time)}: ${r.available ? "available" : "not available"}${r.seconds != null ? ", " + Math.roun
+ d(r.seconds) + " s" : ""}></span>`).join("")

```

```

200 | |||| | '<span class="muted" style="width:auto;height:auto;background:none">none yet</span>';
201 | || | $("checkBtn").disabled = s.checking; $("checkBtn").textContent = s.checking ? "Checking..." : "Check now";
202 | || | $("found").textContent = s.settings.vero_found ? "Found: " + s.settings.vero_found : "Vero CLI not found automatically: enter its full path (in a terminal: where vero).";
203 | || | $("foot").textContent = "Runs on this computer only (127.0.0.1). The check asks: \"Reply with exactly: OK\". No answers or secrets are stored.";
204 | || | if (!settingsLoaded) {
205 | |||| | $("sInterval").value = String(iv); $("sModel").value = s.settings.check_model;
206 | |||| | $("sTimeout").value = s.settings.timeout_seconds; $("sPath").value = s.settings.vero_path; settingsLoaded = true;
207 | || | }
208 | }
209 | function tick(){
210 | || | if (!S) return;
211 | || | const n = S.next_check;
212 | || | if (S.checking) { $("next").textContent = "now"; return; }
213 | || | if (!n) { $("next").textContent = "-"; return; }
214 | || | const s = Math.max(0, Math.round(n - Date.now()/1000));
215 | || | $("next").textContent = s < 60 ? `in ${s} s` : s < 3600 ? `in ${Math.ceil(s/60)} min` : "at " + clock(n);
216 | }
217 | async function load(){
218 | || | try {
219 | |||| | const r = await fetch("/api/state", {cache: "no-store"});
220 | |||| | S = await r.json(); offline = false; render();
221 | || | } catch (e) {
222 | |||| | offline = true; $("hero").className = "card hero"; $("lamp").textContent = "!";
223 | |||| | $("title").textContent = "Vero Status is not running"; $("meta").textContent = "Double-click Vero Status to start it.";
224 | || | }
225 | || | clearTimeout(timer); timer = setTimeout(load, S && S.checking ? 1500 : 5000);
226 | }
227 | async function post(path, body){
228 | || | return fetch(path, {method: "POST", headers: {"Content-Type": "application/json"}, body: body ? JSON.stringify(body) : undefined});
229 | }
230 | $("checkBtn").addEventListener("click", async () => { $("checkBtn").disabled = true; await post("/api/check"); load(); });
231 | $("setBtn").addEventListener("click", () => { const p = $("settings"), open = !p.classList.contains("open");
232 | || | p.classList.toggle("open", open); $("setBtn").setAttribute("aria-expanded", open); if (open) p.scrollToView({behavior: "smooth"}); });
233 | $("saveBtn").addEventListener("click", async () => {
234 | || | const body = {interval_minutes: +$("sInterval").value, check_model: $("sModel").value.trim(),
235 | |||| | timeout_seconds: Math.round(+$("sTimeout").value), vero_path: $("sPath").value.trim().replace(/^"|$/g, "");
236 | || | const r = await post("/api/settings", body), d = await r.json().catch(() => ({}));
237 | || | const m = $("saveMsg"); m.className = "msg " + (r.ok ? "ok" : "err"); m.textContent = r.ok ? "Saved." : (d.error || "Not saved.");
238 | || | if (r.ok) { settingsLoaded = false; load(); }
239 | || | });
240 | $("quitBtn").addEventListener("click", async () => {
241 | || | if (!confirm("Stop Vero Status? Checks stop until you start it again.)) return;
242 | || | try { await post("/api/quit"); } catch (e) {}
243 | || | clearTimeout(timer); $("stopped").classList.add("show");
244 | || | });
245 | setInterval(tick, 1000);
246 | load();
247 | })();
248 | </script>
249 | </body>
250 | </html>
=== END FILE: vero_status/web/index.html ===

```

```
=== VERIFY SCRIPT: verify_rebuild.py (43 lines) ===
```

```
1 | import hashlib, pathlib, sys
2 | # usage: py verify_rebuild.py C:\Tools\vero-status
3 | root = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "vero-status")
4 | CRLF = ['START_HERE.txt', 'scripts/vero.cmd']
5 | FILES = {
6 |     '.gitattributes': 'efa38e63b1747daed572feeff39747d40fd4cfa3a4568b11815cb0bf24332d96',
7 |     '.gitignore': 'b592d17a523d7d8d8bd950419afdeb5f502d4aedd8b65a2887bd71f0c514aa33',
8 |     'CHANGELOG.md': 'ffa4ff4eef221d1e49d7726d5310e8c1e87ec56bdf0e5d6ece6694b11ef5a5dd',
9 |     'README.md': 'd8b413eae23a6ecad126fcb99da6af6dda6912e1f37cd0983fe75b850b683970',
10 |    'START_HERE.txt': 'e3e61140954e96168ca25e366ccb9384c45ef2a66fc04baa2f845eaf2da4ab25',
11 |    'VERO_CONTEXT.md': '057690356ec1dc3d26ce9fe9384bf82fcbbb79c5b721158e9a1e1a452addfa50',
12 |    'Vero Status.pyw': 'd225f8171b6196c264964f66cba8c2992695a171db6c2f047f9c694fb8358197',
13 |    'pyproject.toml': '0490be6ed05d10f4fe955a49fe965dd14698e32c6292a0070776eac1b54f9fd4',
14 |    'scripts/build_source_pdf.py': '1150c226ffe2449688aedc50c56a1404a28958e62668dced9418ea684cb58d91',
15 |    'scripts/fake_vero.py': '4908eb33e4a64621f955e551ef0efce2e644ca3a6ba51e9c202b80cf01d35ae8',
16 |    'scripts/vero': '45890958a3fcde4e02f1d4c41412869e6f71a9efe2d2d237cd54cf8d84e001fa',
17 |    'scripts/vero.cmd': '15e9a293d205d911b2e2746af1c4361906c0b4138dc111be5acc2a653a6cd0ca',
18 |    'tests/__init__.py': '8d8ab1348161355b2c6856609575f5becac99c397071c7dcdad95d00597f39fb',
19 |    'tests/helpers.py': 'dfd00de3c741b7782d8ce5eeaad15ac3534a69b8a7b5dfcbe3fb9d26056ac971',
20 |    'tests/test_monitor_server.py': '4d0c221bbf6b72efae3d736d309368109978227ef2d448295efa034c6f553f75',
21 |    'tests/test_settings.py': '43a3cbf79366e7f16dce3847e58d5b74df54570b570c6e3408c95961554c1303',
22 |    'tests/test_vero.py': '00f1697582183a59e1b0c852dd3f37bbd6fa8d4a0df42ae1294d3b2cde09a44e',
23 |    'vero_status/__init__.py': '8770ca38a34434db168778fe3e48726bf0c041c78e757c1b2df1f37fdf7e1e1a',
24 |    'vero_status/app.py': 'e1149390fa4df886d89d37f0df6f4be93ca73f383e47b8e6b8196711764641b0',
25 |    'vero_status/monitor.py': '2b353f1c385eb006eee7e7114b3daac103140fc647f4f04377bf86626db28472',
26 |    'vero_status/server.py': '4fe22178766d0357e85bb164bbcfbac66555c8914a9b8b68b026e2365c3cc851',
27 |    'vero_status/settings.py': '61d2441d9c9da8c04e624c815a299333724c87786eb6f8b3b6fe646f19beb4',
28 |    'vero_status/vero.py': '31fa059c4bf44a2dadca0ab171b4384bbe4d98ec1a0e6428a13bac5d18d17025',
29 |    'vero_status/web/index.html': '570767ef4b1a040b8ea654d2427b4035ee878cb4b3b32c0fc04afba2e28e2561',
30 | }
31 | bad = 0
32 | for path, sha in FILES.items():
33 |     p = root / path
34 |     if not p.exists():
35 |         print("MISSING", path); bad += 1; continue
36 |     text = p.read_bytes().decode("utf-8").replace("\r\n", "\n")
37 |     if hashlib.sha256(text.encode("utf-8")).hexdigest() != sha:
38 |         print("DIFFERENT", path); bad += 1; continue
39 |     want = text.replace("\n", "\r\n") if path in CRLF else text
40 |     if p.read_bytes() != want.encode("utf-8"):
41 |         p.write_bytes(want.encode("utf-8")); print("fixed line endings", path)
42 |     print("OK: all files match" if not bad else str(bad) + " file(s) wrong: recreate them from the document")
43 | sys.exit(1 if bad else 0)
```

```
=== END VERIFY SCRIPT ===
```